

Data Lakehouse

Ein gemeinsames Architekturverständnis



Agenda

Worum geht's?



Übergeordnetes Ziel ist klar:

- Wir wollen sämtliche Datentransformationen aus der alten SSIS-Welt ablösen.

Für den Weg dahin fehlen jedoch Guidelines:

- Wo soll welche Logik implementiert werden (wo im Projekt?, aber auch: in welchem Layer?)?
- Mit welcher Technologie (pyspark, dbt...)?
- Und wieso eigentlich so und nicht anders?

Problemstellung

Überblick

- **Unklare Verantwortlichkeiten:** Aufgaben der verschiedenen Layer sind nicht definiert. Folglich passieren unterschiedlich tiefgreifende Datentransformationen mal in dem einen und mal in einem anderen Layer.
- **Inkonsistente historische Speicherung von Rohdaten:** Teilweise werden Quelldateien im Archiv der LZ abgelegt, aber für DB-Ingestions existiert kein historisch strukturierter Datenimport.
- **Technischer Mix:** Pyspark, dbt; Delta-, Parquet-, Quellformate; CSV- oder YAML-Konfiguration. Wann und wo sollte ich welche Technologie einsetzen?

Problemstellung

Warum sind die genannten Punkte überhaupt problematisch?

Unklare Verantwortlichkeiten

- **Keine Garantien:** Was kann ich von allen Datasets in z.B. HIS erwarten? Datum immer auf ISO-Format normalisiert, Währung immer Euro, Ganzzahlen immer INT-Typ? => im Moment nicht konsistent
- **Codelokalisierung:** Wo finde ich Standardisierungslogik im Projekt? Neue Entwickler:innen haben es schwer, wo oder gar ob sie entsprechende Logik noch zu implementieren ist.

Unvollständige Rohdaten

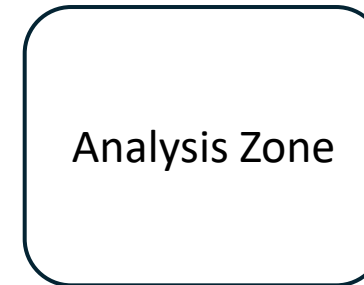
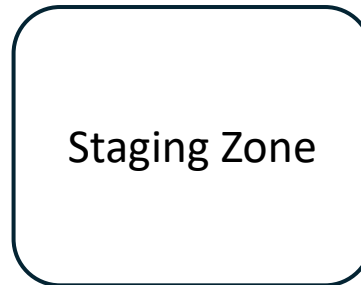
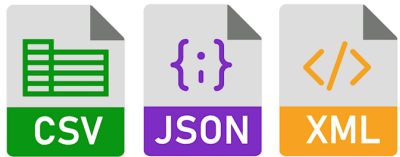
- **Lückenhafte Historie:** Für einige Datei-Quellen können historische Datenimporte im LZ-Archiv nachvollzogen werden, für die meisten DB-Imports nicht. => Das verhindert viele Analysen (Trends, Retros, Vorher/Nachher).
- **Kein konsistentes Reprocessing:** Ohne historisierten, strukturierten Datenimport können spätere Änderungen an Business-Logik oder Datenmodellen nicht mehr einfach „rückwärts“ angewandt werden.

Technologiemix

- **Kognitive Last:** Je mehr Varianten ohne Guideline, desto mehr Spezialwissen ist nötig. Bus-Faktor steigt.
- **Redundanzen:** Tools lösen ähnliche Probleme. Ohne klare Leitplanken kommt es zur Duplikation von Logik in mehreren Technologien.

Problemstellung

Wie sieht unsere Architektur gerade aus?

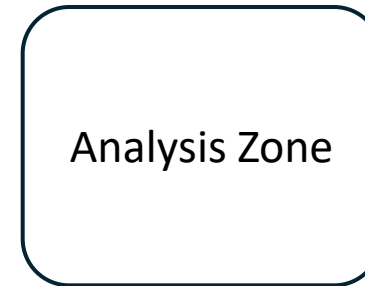
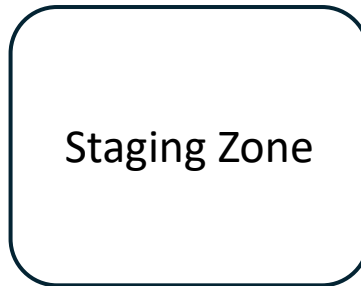
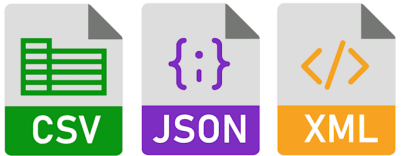


PowerBI



Problemstellung

Herausforderungen in der Landing Zone



- Oft manuelles Kopieren von Quelldateien auf LZ
- Zwei Verantwortlichkeiten:
 1. Temporärer Ablageort für Daten verschiedener Quellsysteme
 2. Archiv für verarbeitete Datasets

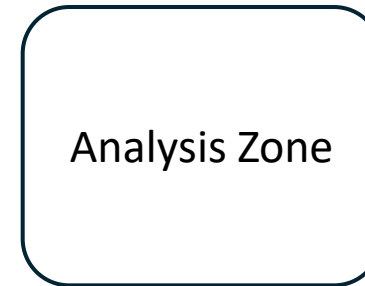
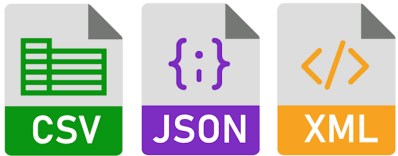


PowerBI



Problemstellung

Herausforderungen in der Staging Zone



- Nur sporadisch verwendet, z.B. nicht bei ABR1 oder ABR2
- Datentypkonvertierung wird durchgeführt, aber nicht in ein einheitliches Format (Parquet und Delta)

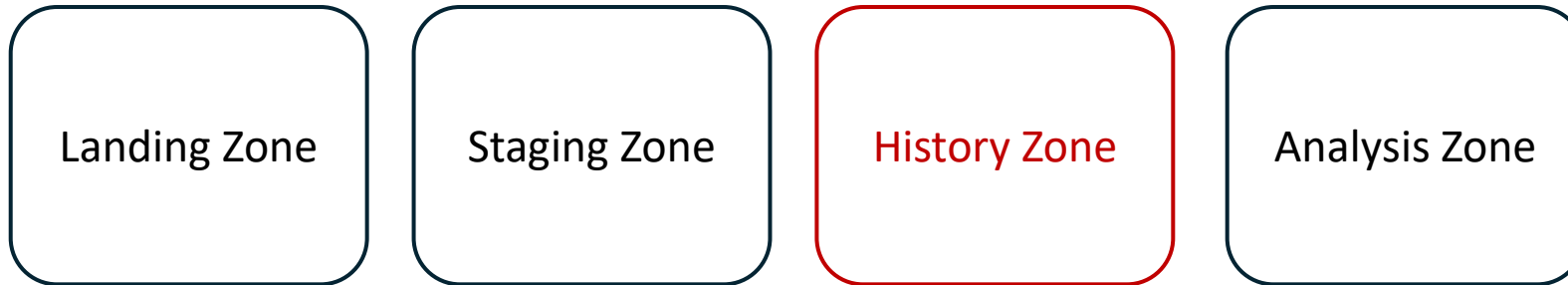
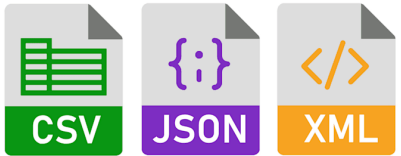


PowerBI



Problemstellung

Herausforderungen in der History Zone



- Manchmal Standardisierung von Daten (wie Währungsvereinheitlichung bei VOM), oft aber auch nicht
- Sowohl Transformation mit pyspark als auch mit dbt
- Oft vollständige Partition-Rewrites anstelle von Updates

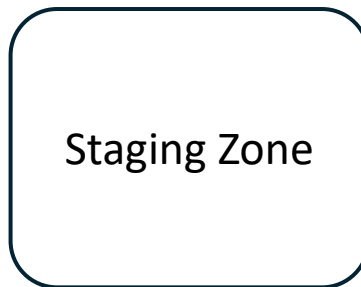
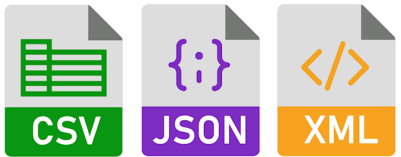


PowerBI



Problemstellung

Herausforderungen in der Analysis Zone



- Größtenteils noch nicht abgelöst, also noch als SSIS-Paket implementiert



PowerBI



Problemstellung

Herausforderungen im data-pipelines-Projekt

```
data-pipelines/
|
|─ src/data_pipelines/ # Main project package
|   |─ config/         # Configurations/secrets, should be moved to Vault
|   |─ his_ana/        # Currently empty, but should be implemented as a dbt project
|   |─ lz_his/         # Also does ingestion into our lakehouse but should ideally mainly do schema enforcement
|   |─ src_lz/         # Absolutely empty, but we are doing ingestion ofc. Does our data magically spawn in our LZ?
|   |─ utils/          # Utility functions (logging, config, key store, etc.)
|
|─ pyproject.toml      # Defines project dependencies
```

Zusätzliche Fragestellungen:

- Wo soll das dbt-Projekt in dieser Struktur angesiedelt werden? Welche Transformationen sollen mit dbt implementiert werden, welche mit pyspark?
- Wir haben oft ähnliche (SELECT *) Ingestion-Logik. Wie verhindern wir Code-Duplikation?
- Was ist zulässige/typische lz_his-Logik? Welche Transformationen sollten erst von HIS nach ANA passieren?

Ein Weg zur Lösung

Eine gemeinsame Vision



Data Lakehouse

Ein gemeinsames Vokabular

Ein Data Lakehouse ist eine Datenmanagement-Architektur, die es Organisationen ermöglicht, **rohe, semi-strukturierte und strukturierte Daten** in einem einzigen System zu speichern und gleichzeitig transaktionale Garantien (**ACID**), Schema-Validierung, **Governance** sowie die für Analytik und Machine Learning erforderlichen Performance-Optimierungen bereitzustellen.

Data Lakehouse

Eigenschaften

- **Einheitlicher Speicher:** Strukturierte (Tabellen), semi-strukturierte (JSON) und unstrukturierte Daten (Logs) alle an einem Ort.
- **Transaktionen möglich:** ACID-Unterstützung und paralleler Lese- und Schreibzugriff durch Open Table Formate wie Delta.
- **Schemaverwaltung & Governance:** Schemaevolution, Validation, Metadatenkataloge, Lineage und Zugriffskontrolle.
- **Performance:** Indizes, Statistiken, Caching, Clustering, vektorisierte Abfragen => DWH-Performance
- **Multimodale Analyse:** BI/SQL-Reporting, aber auch Machine Learning Pipelines auf denselben Daten möglich ohne Duplizierung

Data Lakehouse

Aber haben wir unsere Architektur nicht Data Lake genannt?

Ein **Data Lake** ist flexibel und kostengünstig (Schema-on-Read), hat jedoch historisch ACID-Garantien, Governance und konsistente Abfrageperformance vermissen lassen.

Ein **Data Lakehouse** behält den offenen und günstigen Speicher bei, ergänzt jedoch Transaktionen, Schema-Validierung, Kataloge und Indexierung
→ dadurch werden zuverlässige Analysen auf denselben Daten möglich.

Data Lakehouse

Und was ist jetzt anders als früher in unserem Data Warehouse?

Früher bot ein Data Warehouse starke Governance und schnelle SQL-Abfragen, war jedoch starr und primär auf strukturierte BI ausgerichtet. Ein Lakehouse bringt vergleichbare Verwaltung und Performance in offene Formate auf Objektspeicher und unterstützt semi-/unstrukturierte Daten sowie ML – ohne redundante ETL-Prozesse.

ETL vs. ELT

Schreiben wir aktuell immer noch ETL-Pipelines? (Nein.)

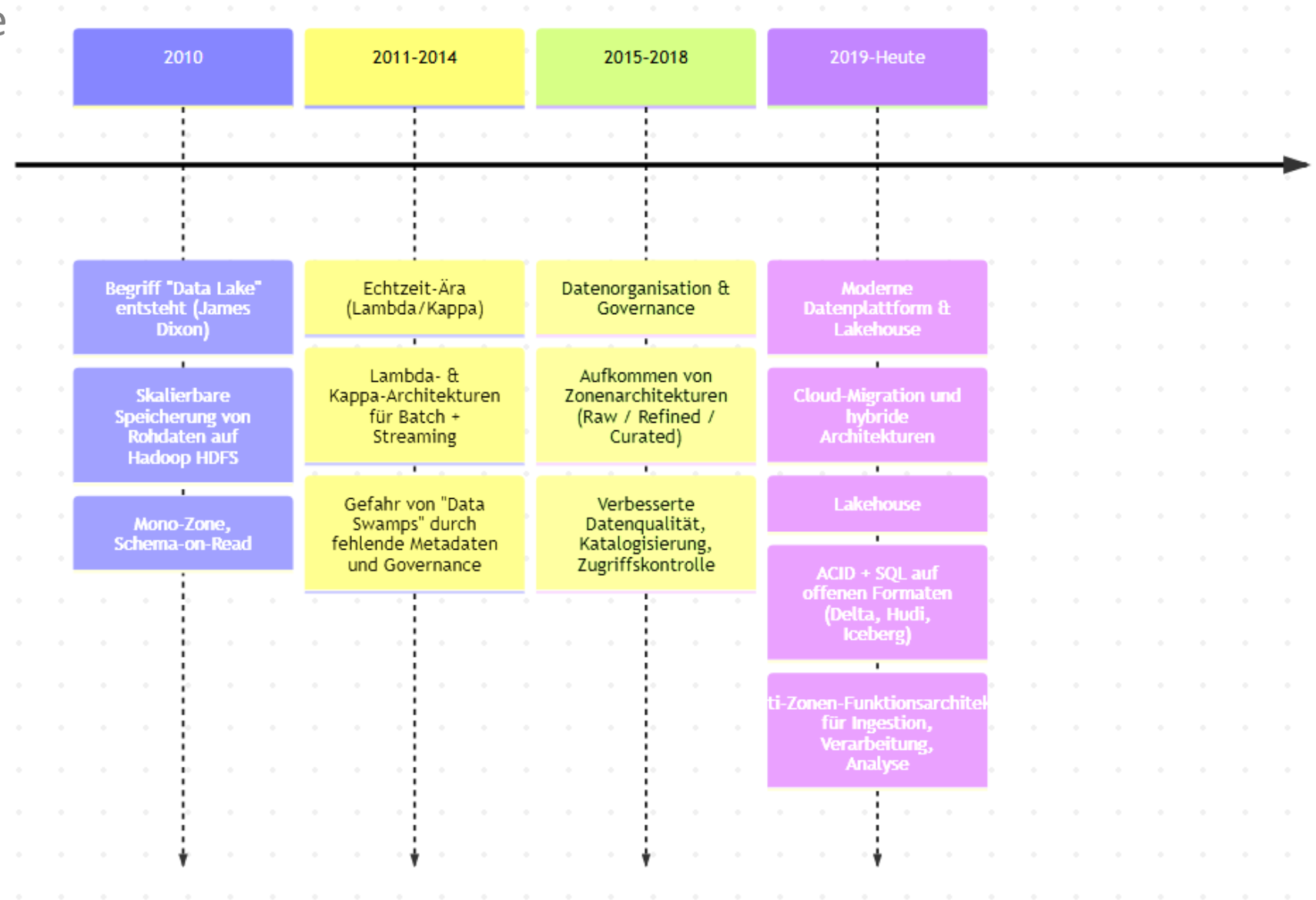
ETL bedeutet, dass Daten vor dem Laden in den analytischen Speicher transformiert werden (klassischer Data-Warehouse-Ansatz).

ELT hingegen lädt Daten zuerst ins Lakehouse (raw) und transformiert sie anschließend **in-place** mit skalierbaren Engines wie Spark, Flink oder Trino.

Das **Lakehouse** bevorzugt **ELT**, dennoch setzen wir ETL-ähnliche Prüfungen (Quality Gates) zwischen den Layers.

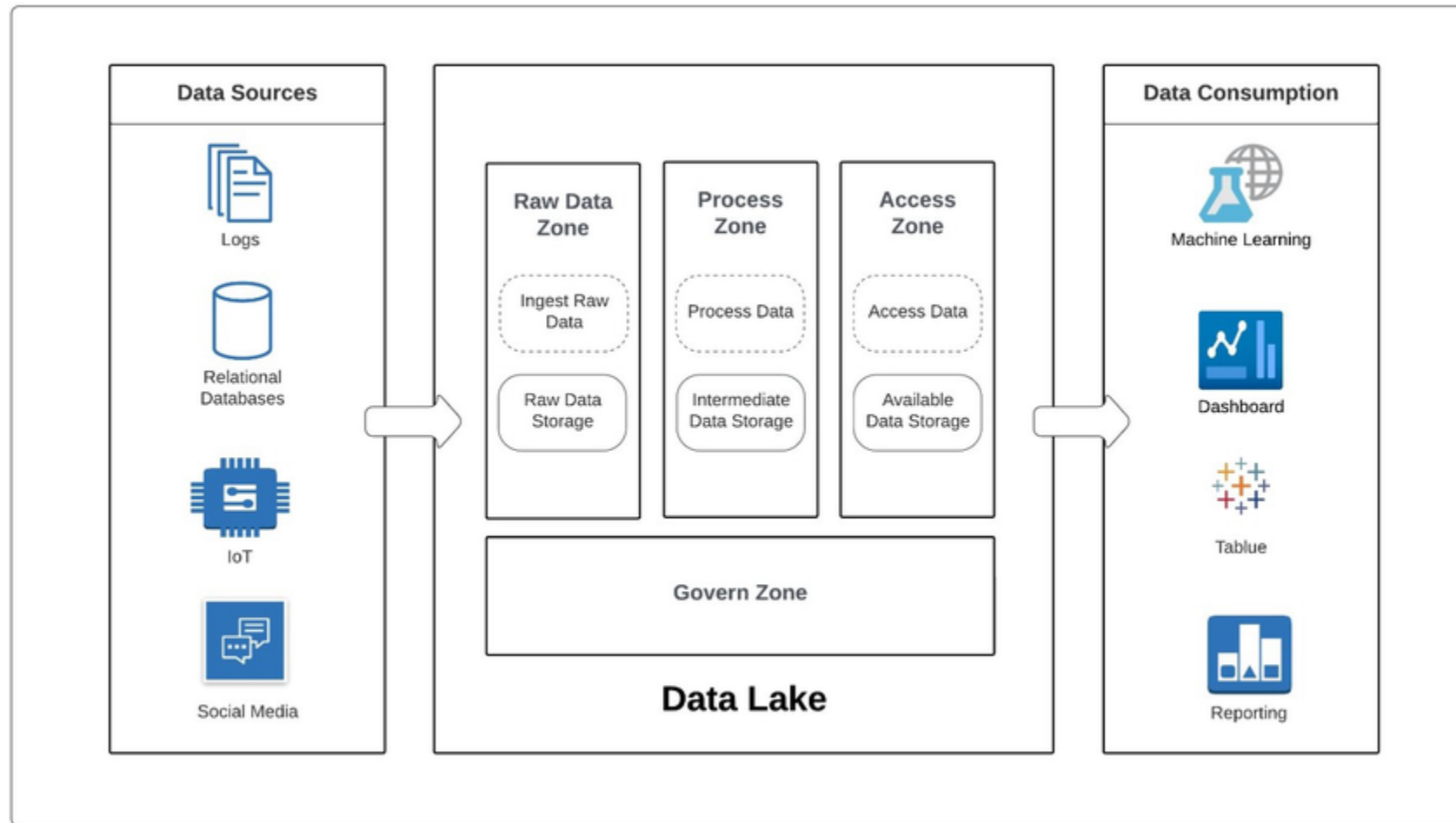
Architekturen

Eine kurze Historie



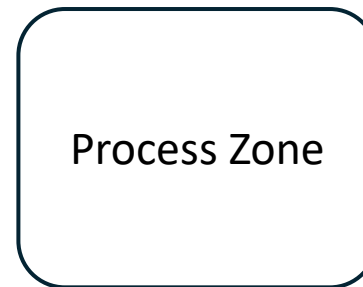
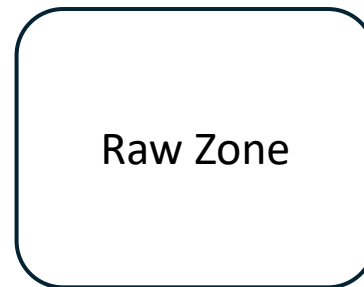
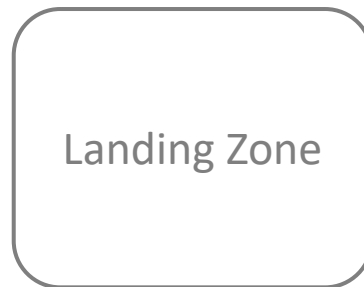
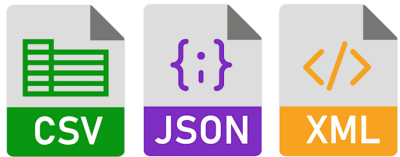
Multi-Zone-Functional

Ein moderner Architekturansatz



Multi-Zone-Functional

Ein neuer Architekturvorschlag



PowerBI



Multi-Zone-Functional

Landing Zone: Idealerweise unnötig

Transient Landing Zone (Bucket: lz)

Hinweis: Diese Zone ist **optional**. Bevorzugt wird ein Muster, bei dem Daten direkt aus den Quellen in die Raw Zone importiert werden. Wenn das nicht möglich ist, können Dateien manuell in die Transiente Landing Zone gelegt werden, damit sie von Ingestion-Jobs aufgenommen werden. In diesem Sinne kann die Transiente Landing Zone als externes Quellsystem verstanden werden, das das Lakehouse speist.

Zweck: Kurzlebiger, **temporärer Ablagebereich** für neu importierte Dateien oder Streams vor der Validierung. Daten sollten von hier schnell in die Raw Zone verschoben oder verworfen werden.

Vertrag

Mutationen: Append only; automatische Bereinigung nach konfigurierbarer Aufbewahrungszeit (z. B. 1–7 Tage).

Schema: Keine Erzwingung; Dateien werden in der Form gespeichert, in der sie ankommen.

Qualität: Keine.

Erlaubt: Dateiablage.

Verboten: Langfristige Speicherung, Transformationen, Business-Logik.

Retention: Kurzlebig; automatische Löschung, sobald die Daten in die Raw Zone importiert wurden.

Multi-Zone-Functional

Raw Zone: Unser neues History Layer

Raw Zone (Bucket: his → raw)

Zweck: Langfristige Historisierung von Rohdaten aus den Quellsystemen. Dient als unveränderliches System of Record und bewahrt Daten exakt so auf, wie sie geliefert wurden – für Nachvollziehbarkeit und Reproduzierbarkeit.

Vertrag

Mutationen: Append-only. Neue Dateien oder Batches können hinzugefügt werden; Metadaten dürfen zur besseren Rückverfolgbarkeit angereichert werden. Keine semantischen Änderungen oder Löschungen von Datensätzen.

Schema: Schema-on-Read; Daten werden im gelieferten Format gespeichert.

Qualität: Minimale technische Validierung (Parsbarkeit, grundlegende Typprüfungen, Dateiexistenz, Größe > 0, Prüfsumme). Keine fachlichen Regeln.

Erlaubt: Datei-Lifecycle-Management; Metadatenanreicherung wie Dateiname, Pfad, Größe, Endung, Quelle, Erstellungsdatum; Quarantäne defekter Objekte.

Verboten: Joins, Aggregationen, Standardisierung, destruktive Updates.

Retention: Lang (10 Jahre oder dauerhaft).

In den folgenden Zonen wird die Retention nicht erneut erwähnt, da sie der Raw Zone ähnelt.

Multi-Zone-Functional

Process Zone: Sicherstellen von Datenqualität

Process Zone (Bucket: process)

Zweck: In dieser Zone werden Daten **transformiert und Datenqualität gewährleistet**, und alle während dieser Transformationen entstehenden Zwischendaten werden hier gespeichert. Dies umfasst Batch- und/oder Echtzeitoperationen.

Vertrag

Mutationen: Diese Zone ist geprägt von **Standardisierung**. Dazu gehören **Formatvereinheitlichung**, ebenso **Bereinigung und Anreicherung** von Daten. Daten aus mehreren Quellen können **integriert** werden, und die Zone verwaltet die **Schema-Evolution**, um sich im Zeitverlauf an Änderungen anzupassen. Es entstehen Zwischendatensätze, und bestehende verarbeitete Daten können im Zuge iterativer Verfeinerungen aktualisiert oder überschrieben werden.

Schema: Schema-Erkennung und -Validierung sowie **Formatvereinheitlichung** auf optimierte, selbsterklärende Formate wie **Delta Lake**, die robuste Schema-Management-Funktionen bereitstellen.

Qualität: Strenges Data-Quality-Management ist ein zentrales Ziel. Dazu gehören das **Erkennen und Beheben von Problemen** wie fehlenden Werten, Inkonsistenzen, Schreibfehlern und Redundanzen. **Deduplikation** und **zusätzliche Compliance- und Qualitätschecks** werden durchgeführt, um Datenzuverlässigkeit sicherzustellen. Validierungsregeln werden abgeleitet und angewendet, um hohe Datenstandards einzuhalten.

Erlaubt: Standardisierung (z.B. Daten, Währungen), Datenbereinigung (Deduplikation, fehlende Werte, Inkonsistenzen), Dataset-unabhängige Logik, Integration mehrerer Quellen, Umgang mit Schema-Evolution, Erfassung von Provenienz, Formatoptimierung (z. B. Parquet/Delta) sowie Speicherung von Zwischendatensätzen.

Verboten: Dataset-spezifische Fachlogik, verbraucher-spezifische Denormalisierungen, Änderungen, die Granularität oder fachliche Bedeutung verändern, willkürliche Löschungen, unveränderte Rohdaten.

Multi-Zone-Functional

Access Zone: Unsere neue Analysis Zone

Access Zone (Bucket: ana → access)

Zweck: Ermöglicht Self-Service-Datenverbrauch für Reporting, BI, Analytics und ML. Speichert verbraucheroptimierte Produkte – denormalisierte und/oder aggregierte Datensätze wie Fakten-/Dimensionsmarts, Wide Tables, Feature Sets und Serving Indexes.

Vertrag

Mutationen: Sämtliche Datentransformationen gemäß der Nutzer, Vollständige Neuaufbauten oder inkrementelle Aktualisierungen. Schemas und Transformationslogik sind versioniert; für Breaking Changes gelten Auslaufzeiträume.

Schema: Verbraucherorientiert mit dokumentierten Metrikdefinitionen, fachlicher Granularität und Service Level Objectives (SLOs).

Qualität: Tests stellen Metrikkonsistenz, dimensionale Integrität mit konformen Upstream-Entitäten und Einhaltung von Freshness-SLAs sicher.

Erlaubt: Dataset-spezifische Fachlogik, Aggregationen, **Denormalisierungen** (Star-Schema), Window-Berechnungen, Feature Engineering und Performance-Optimierungen (Statistiken, Clustering/Compaction).

Verboten: Rückkopplung in Upstream-Zonen (keine Reverse Dependencies zu Process/Raw) außer über dokumentierte Reconciliation-/Backfill-Verfahren.

Versionierung & Rollout: Verbraucherprodukte (Marts/Features) tragen semantische Versionen (z. B. _v2). Alte und neue Versionen können während der Auslaufphase parallel betrieben werden; Änderungsprotokolle werden für Downstream-Konsumenten veröffentlicht.

Multi-Zone-Functional

Was bedeutet das konkret für die aktuelle Architektur?

Landing Zone & Staging Zone

- **Verhindern**, wenn möglich => automatische Ingestion bevorzugen
- Incoming/archive-Struktur auflösen -> nur als **temporären Ablageort** verwenden

History Zone

- Umbenennen in **raw-zone**
- In **Quellformaten** historisieren (anstatt Mix aus Parquet/Delta)
- Daten einheitlich mit **pyspark** importieren
 - **Keine Datentransformationen** durchführen

Analytics Zone

- Aufteilen in **process-zone** und **access-zone**

Process Zone

- Speichern in **Delta**-Format
- Verarbeitung mit **pyspark**
 - Nur benötigten Datenstand behalten
 - Minimale Datentransformationen (d.h. technische **Standardisierung**)

Access Zone

- Speichern in **Delta**-Format
- Verarbeitung mit **dbt**
 - Fachliche Datentransformationen
- **Reporting-ready**

Beispiel: Voucher-Tabelle

Ein Controlling-Dataset auf Durchreise



Tabelle: Voucher

- Zweck: Enthält **Controlling-Belege** mit Geschäftsdatusfelder, Kostenobjekten, Konten, Werte sowie beschreibende Texte für Controlling-KPIs.
- Datenumfang: ca. 4,5 Mio. Zeilen (~ 1 GB)

Beispiel: Voucher-Tabelle

Raw Zone: Eigentlich nur ein SELECT *

Tabelle: Voucher

- **Zweck:** Enthält **Controlling-Belege** mit Geschäftsdatumsfelder, Kostenobjekten, Konten, Werte sowie beschreibende Texte für Controlling-KPIs.
- **Datenumfang:** ca. 4,5 Mio. Zeilen (~ 1 GB)

Täglicher Import
via
Airflow/Pyspark



Full Load
(für Einfachheit,
behalte 7 Tage)

```
s3://raw-zone/controlling/voucher/
├── data/
│   ├── load_date=2025-11-14/
│   │   ├── voucher_20251114T103015Z_part-00001.jsonl.gz
│   │   ├── voucher_20251114T103015Z_part-00002.jsonl.gz
│   │   └── voucher_20251114T103015Z_part-00003.jsonl.gz
│   ├── load_date=2025-11-15/
│   │   ├── voucher_20251115T101200Z_part-00001.jsonl.gz
│   │   └── voucher_20251115T101200Z_part-00002.jsonl.gz
└── metadata/
    ├── load_date=2025-11-14/
    │   └── metadata.json
    └── load_date=2025-11-15/
        └── metadata.json
```

metadata.json

Enthält technische Metadaten pro Ladedatum: Extraktzeitpunkt, Quelle (@controlling), verwendete View, Anzahl Zeilen, Liste der erzeugten Dateien (mit Größe & Checksum). Keine fachlichen Informationen; dient nur der Nachvollziehbarkeit und Integritätsprüfung im Raw Layer.

Beispiel: Voucher-Tabelle

Process Zone: Sicherstellen der Datenqualität

s3://raw-zone/controlling/voucher/

data/

load date=2025-11-14/

voucher_20251114T103015Z_part-00001.jsonl.gz

voucher 20251114T103015Z part-00002.json.gz

voucher 20251114T103015Z part-00003.json.gz

load date=2025-11-15/

voucher 20251115T101200Z part-00001.json.gz

voucher 20251115T101200Z part-00002.jsonl.gz

metadata/

load date=2025-11-14/

```
metadata.json
```

load date=2025-11-15/

metadata.json

Angestoßen nach Fertigstellung der Ingestion



MERGE Updates via pyspark

s3://process-zone/controlling/voucher/

_delta_log/

```
000000000000000000000000.json
```

```
0000000000000000000000001.json
```

...

part-00000-uuid.snappy.parquet

part-00001-uuid.snappy.parquet

part-00002-uuid.snappy.parquet

Technische Standardisierung

Julian-Datumsfelder → echte Date-/Timestamp-Typen

Cent-Beträge → numerische DECIMAL-Werte

Technische Metadaten hinzufügen: source, load_date, load_timestamp, time_period

Quellstruktur weitgehend beibehalten; **keine** fachlichen Umbenennungen

Nur minimale technische Filter (z. B. nicht parsierbar)

Speicherung als **Delta Lake**; Tabelle zeigt **aktuellen Zustand** via MERGE (keine Historisierung)

Beispiel: Voucher-Tabelle

Access Zone: Bereit für Berichte/Self BI (Cubes)

```
s3://process-zone/controlling/voucher/  
_delta_log/  
  00000000000000000000.json  
  00000000000000000001.json  
  ...  
part-00000-uuid.snappy.parquet  
part-00001-uuid.snappy.parquet  
part-00002-uuid.snappy.parquet
```

Angestoßen nach
Fertigstellung des
Processing



Konvertierung in Stern-
Schema via dbt

Fachliches Datenmodell

Fachliche Datumsfelder + Schlüssel (*_SK) erzeugen

Periodenfelder bilden (PERIODE)

Surrogate Keys (MD5) erzeugen

Umbenennung in fachliche deutsche Spaltennamen

Fachliche Filter anwenden (Mandanten-Blacklist, leere Konten, Wertart, Beleg-Char)

Fachlich aufbereitete Beträge (BETRAG_*) erzeugen

Hilfsspalten wie DMSKEY generieren

Ergebnis: voll fachlich nutzbares Delta-Fact-Table

```
s3://access-zone/controlling/  
fact/  
  belege/  
    _delta_log/  
      00000000000000000000.json  
      00000000000000000001.json  
      ...  
    part-00000-uuid.snappy.parquet  
    part-00001-uuid.snappy.parquet  
    part-00002-uuid.snappy.parquet  
  
dim/  
  kostenart/  
    _delta_log/  
    part-*.parquet  
  
  controllingobjekt/  
    _delta_log/  
    part-*.parquet  
  
  ...
```

Nutzen jeder Zone

Retrospektive

Raw Zone

Unverändert. Vollständig. Historisiert.

Quelle 1:1 gespeichert

Perfekt für Audits, Forensik, Reprocessing

Grundlage für langfristige Analysen & Data Science

Process Zone

Technisch bereinigt. Getypt. Aktueller Zustand.

Standardisierte Datentypen

Einheitliches, sauberes Integrationsmodell

Stabiler Einstiegspunkt für alle Downstream-Marts

Access Zone

Fachlich modelliert. Reporting-ready.

Surrogate Keys, Periodenlogik, Business-Filter

Deutsche Spaltennamen, saubere Facts & Dims

Direkt nutzbar für BI, KPIs, Controlling & Analytics

Ausblick

Wie kommen wir vom aktuellen zum Zielzustand?

Neue Namen für Buckets

*Neue Verantwortlichkeiten.
Neue Namen.*

- **raw:** Hier kommt das Archiv aus der lz hin.
- **process:** Da in der his zukünftig nicht mehr historisiert wird, ist Umbenennung notwendig.
- **ana:** Kann beibehalten werden, da dort Analytics-Logik passiert.

Isolation der Ingestion

*Entwirren von
Transformationsschritten.*

- Ingestion passiert gerade in src -> his, teilweise mit Datentransformationen.
- Für ein sauberes System of Record wollen wir Daten so quellnah wie möglich in raw speichern.
- Ingestion-Logik kann zunächst parallel zu produktiven Pipelines isoliert werden.

Processing Zone

Standardisierung an einem Ort.

- Auch die Processing Zone kann zunächst als neuer Bucket parallel betrieben werden.
- Für ein sauberes System of Record wollen wir Daten so quellnah wie möglich in raw speichern.
- Ingestion-Logik kann zunächst parallel zu produktiven Pipelines isoliert werden.

Analytics/Access Zone

Schritt für Schritt.

- In der letzten Zone ana passieren Fachlogik und Denormalisierung, d.h. Optimierung für Reporting/Self-BI.
- Data Product für Data Product kann hier die vorherige Zone von der alten his auf die neue process Zone umgestellt werden.

Abschließende Worte

Was diese Präsentation ist und was nicht

- + **Intro in Lakehouse-Architekturen:** Übersicht über Architekturen in der Literatur und Einführung in Best Practices.
- + **Eine persönliche Empfehlung:** Basierend auf meinen Recherchen und Unimodulen letzter Semester halte ich diese Änderungen für erstrebenswert.
- + **Diskussionsgrundlage:** Diese Präsentation soll Input für Diskussionen über Standards, Prozesse und gerne auch Detailfragen geben.
- **Der einzig richtige Weg:** Sowohl historisch als auch aktuell ist die Big Data Architekturlandschaft vielseitig und auch andere Ansätze können genauso gerechtfertigt sein.
Allerdings: Ohne klare Definitionen/Richtlinien kann es nicht weitergehen.
- **Ausschließlich meine Meinung:** Die Inhalte basieren auf umfangreichen Literaturrecherchen und basieren auf dem Stand der Forschung und Praxis.
- **Eine verbindliche Vorgabe:** Ziel ist es kollaborativ Konventionen/Standards auszuarbeiten, an die sich alle halten. Dieser Input sollte als erster Entwurf/Blueprint verstanden werden, mit dem wir starten können.

“Coming together is a beginning; keeping together is progress; working together is success.” – Henry Ford



Anhang

Ein paar Extras

Struktur der Metadaten

Ein Beispiel für die Raw Zone

```
{
  "ingest_timestamp": "2025-08-29T08:15:04Z",
  "file_count": 2,
  "total_bytes": 14500213,
  "tables": [
    {
      "name": "table1.json", "bytes": 7340021, "sha256": "...", "check_parsable": true, "check_nonempty": true,
      "schema": {
        "columns": [
          { "name": "<col1>", "type": "<type>" },
          { "name": "<col2>", "type": "<type>" }
        ]
      }
    },
    {
      "name": "table2.json", "bytes": 7160192, "sha256": "...", "check_parsable": true, "check_nonempty": true,
      "schema": {
        "columns": [
          { "name": "<col1>", "type": "<type>" }
        ]
      }
    }
  ],
  "checks_passed": true
}
```

Ausnahmen

Theorie in der realen Welt

No one shape fits it all: In begründeten Fällen kann von den Vorgaben für die einzelnen Zonen abgewichen werden. Dies muss allerdings hinreichend dokumentiert werden.

Beispiel: Patientenbildung in der Process Zone

- Klassisches Record Linkage Problem
- Zur Lösung ist Fachlogik notwendig, daher eigentlich in Access Zone verortet
- Allerdings: Lösung nicht trivial/nicht einfach in SQL abbildbar (hybrides ML-Modell, Validierung etc.)
- Zusätzlich: Viele Consumer benötigen den zusammengeführten Datensatz

=> Implementierung in Process Zone mit Python

Referenzen

- * **Azzabi, S., Alfughi, Z., & Ouda, A.** (2024). **Data Lakes: A Survey of Concepts and Architectures**. *Computers*, 13*(7), 183.
<https://doi.org/10.3390/computers13070183>.
- * **Hai, R., Koutras, C., Quix, C., & Jarke, M.** (2023). **Data Lakes: A Survey of Functions and Systems**. *IEEE Transactions on Knowledge and Data Engineering*, 35*, 12571–12590.
- * **Ravat, F., & Zhao, Y.** (2019). **Data Lakes: Trends and Perspectives**. In S. Hartmann et al. (Eds.), *Database and Expert Systems Applications (DEXA 2019), LNCS 11706* (pp. 304–313). Springer Nature Switzerland AG.